



10.2. Python: List Comprehension

"A concise way to create lists"

Previous Sections:

[`</>` 10. Python: Lists](#)

[`</>` 10.1. Python: Common Methods in Lists](#)

Contents:

[Basic Syntax of List Comprehension](#)

[1. Creating a List using a Loop](#)

[2. Creating Squares of Numbers](#)

[3. Applying Conditions using if](#)

[4. Applying if-else in List Comprehension](#)

[5. Creating Lists from Another List](#)

[6. Converting Strings to Uppercase](#)

[7. Nested List Comprehension](#)

[Summary](#)

Up until now, we have been creating lists manually:

```
numbers = [1, 2, 3, 4, 5]
```

This works perfectly fine for small lists.

But imagine we want to create:

- Numbers from 1 to 1000
- Squares of numbers
- Even numbers only
- Modified values from another list

Writing them manually would be extremely inefficient. This is where List Comprehension becomes very powerful.

List Comprehension allows us to create lists using loops directly inside square brackets. It is shorter, cleaner, and often more readable than traditional loops.

In this section, we introduce various ways to create lists using loops, and also apply conditional statements (`if-else`) inside List Comprehension.

Basic Syntax of List Comprehension

General syntax:

```
[expression for variable in iterable]
```

Think of it like this:

```
new_list = []  
  
for variable in iterable:  
    new_list.append(expression)
```

List Comprehension simply compresses everything into one line.

1. Creating a List using a Loop

- Traditional approach:

```
numbers = []

for i in range(1, 6):
    numbers.append(i)

print(numbers)
```

[1, 2, 3, 4, 5]

- Now using List Comprehension:

```
numbers = [i for i in range(1, 6)]

print(numbers)
```

[1, 2, 3, 4, 5]

Much shorter, but still doing the exact same thing.

2. Creating Squares of Numbers

- Traditional loop:

```
squares = []

for i in range(1, 6):
    squares.append(i ** 2)

print(squares)
```

[1, 4, 9, 16, 25]

- Using List Comprehension:

```
squares = [i ** 2 for i in range(1, 6)]  
print(squares)
```

[1, 4, 9, 16, 25]

The expression `i ** 2` is applied to every value generated by the loop.

3. Applying Conditions using if

List Comprehension can also filter elements using conditions.

Example: Extract only even numbers.

- Traditional loop:

```
even_numbers = []  
  
for i in range(1, 11):  
    if i % 2 == 0:  
        even_numbers.append(i)  
  
print(even_numbers)
```

[2, 4, 6, 8, 10]

- Using List Comprehension:

```
even_numbers = [i for i in range(1, 11) if i % 2 == 0]  
print(even_numbers)
```

[2, 4, 6, 8, 10]

Here: The loop generates numbers from 1 to 10. The `if` condition filters only even numbers

4. Applying if-else in List Comprehension

We can also apply full `if-else` logic.

Example: Replace even numbers with "Even" and replace odd numbers with "Odd"

- Traditional loop:

```
result = []

for i in range(1, 6):
    if i % 2 == 0:
        result.append("Even")
    else:
        result.append("Odd")

print(result)
```

['Odd', 'Even', 'Odd', 'Even', 'Odd']

- Using List Comprehension:

```
result = ["Even" if i % 2 == 0 else "Odd" for i in range(1, 6)]
print(result)
```

['Odd', 'Even', 'Odd', 'Even', 'Odd']

Notice something important: The `if-else` expression comes BEFORE the loop.

General syntax:

```
[value_if_true if condition else value_if_false for variable in iterable]
```

5. Creating Lists from Another List

List Comprehension is often used to transform existing lists.

Example:

```
numbers = [1, 2, 3, 4, 5]
double_numbers = [i * 2 for i in numbers]

print(double_numbers)
```

[2, 4, 6, 8, 10]

Every element is modified and stored into a new list.

6. Converting Strings to Uppercase

```
names = ['iris', 'python', 'machine learning']
uppercase_names = [name.upper() for name in names]

print(uppercase_names)
```

['IRIS', 'PYTHON', 'MACHINE LEARNING']

This demonstrates that List Comprehension works with strings too.

7. Nested List Comprehension

We can even use nested loops.

```
pairs = [(x, y) for x in range(1, 4) for y in range(1, 3)]

print(pairs)
```

[(1, 1), (1, 2),
(2, 1), (2, 2),
(3, 1), (3, 2)]

This is similar to:

```
pairs = []

for x in range(1, 4):
    for y in range(1, 3):
        pairs.append((x, y))

print(pairs)
```

```
[(1, 1), (1, 2),
 (2, 1), (2, 2),
 (3, 1), (3, 2)]
```

Summary

List Comprehension is one of the most elegant features in Python.

It allows us to:

- Create lists efficiently
- Transform data quickly
- Apply filtering conditions
- Write cleaner and shorter code

And as we move deeper into Data Science and Artificial Intelligence, List Comprehension becomes extremely common because we constantly process large collections of data.

So while List Comprehension may initially look strange... once we get used to it, it becomes one of the most powerful and beautiful tools in Python.

Hand-On Exercises:

[`</>` Ex 1. Python Exercises](#)

Next Section:

[`</>` 11. Python: Tuples](#)